配置

配置权限

复制页面

使用细粒度权限规则、模式和托管策略来控制 Claude Code 可以访问和执行的操作。

Claude Code 支持细粒度权限，因此您可以精确指定代理允许执行的操作和不允许执行的操作。权限设置可以检入版本控制并分发给组织中的所有开发人员，也可以由个别开发人员自定义。

权限系统

Claude Code 使用分层权限系统来平衡功能和安全性：

工具类型	示例	需要批准	”是，不再询问”行为
只读	文件读取、Grep	否	不适用
Bash 命令	Shell 执行	是	每个项目目录和命令永久有效
文件修改	Edit/Write 文件	是	直到会话结束

管理权限

您可以使用 `/permissions` 查看和管理 Claude Code 的工具权限。此 UI 列出所有权限规则和它们来自的 settings.json 文件。

Allow 规则允许 Claude Code 使用指定的工具而无需手动批准。

Ask 规则在 Claude Code 尝试使用指定工具时提示确认。

Deny 规则防止 Claude Code 使用指定的工具。

规则按顺序评估：**deny -> ask -> allow**。第一个匹配的规则获胜，因此 deny 规则始终优先。

Claude Code Docs

权限模式

Claude Code 支持多种权限模式来控制工具的批准方式。在您的**设置文件**中设置 `defaultMode` ：

模式	描述
`default`	标准行为：在首次使用每个工具时提示权限
`acceptEdits`	自动接受会话的文件编辑权限
`plan`	Plan Mode：Claude 可以分析但不能修改文件或执行命令
`dontAsk`	自动拒绝工具，除非通过 `/permissions` 或 `permissions.allow` 规则预先批准
`bypassPermissions`	跳过所有权限提示（需要安全环境，请参见下面的警告）

> ⚠ `bypassPermissions` 模式禁用所有权限检查。仅在隔离环境（如容器或虚拟机）中使用此模式，其中 Claude Code 无法造成损害。管理员可以通过在**托管设置**中将 `disableBypassPermissionsMode` 设置为 `"disable"` 来防止此模式。

权限规则语法

权限规则遵循格式 `Tool` 或 `Tool(specifier)` 。

匹配工具的所有使用

要匹配工具的所有使用，请使用不带括号的工具名称：

规则	效果
`Bash`	匹配所有 Bash 命令
`WebFetch`	匹配所有网络获取请求
`Read`	匹配所有文件读取

`Bash(*)` 等同于 `Bash` 并匹配所有 Bash 命令。

使用说明符进行细粒度控制

在括号中添加说明符以匹配特定的工具使用：

Claude Code Docs

规则	效果
`Bash(npm run build)`	匹配确切的命令 `npm run build`
`Read(./.env)`	匹配读取当前目录中的 `.env` 文件
`WebFetch(domain:example.com)`	匹配对 example.com 的获取请求

通配符模式

Bash 规则支持带有 `*` 的 glob 模式。通配符可以出现在命令中的任何位置。此配置允许 npm 和 git commit 命令，同时阻止 git push：

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git commit *)",
      "Bash(git * main)",
      "Bash(* --version)",
      "Bash(* --help *)"
    ],
    "deny": [
      "Bash(git push *)"
    ]
  }
}
```

`*` 前的空格很重要：`Bash(ls *)` 匹配 `ls -la` 但不匹配 `lsof`，而 `Bash(ls*)` 两者都匹配。旧版 `:*` 后缀语法等同于 `*` 但已弃用。

工具特定的权限规则

Bash

Bash 权限规则支持带有 `*` 的通配符匹配。通配符可以出现在命令中的任何位置，包括开头、中间或结尾：

Claude Code Docs

- `Bash(npm run build)` 匹配确切的 Bash 命令 `npm run build`
- `Bash(npm run test *)` 匹配以 `npm run test` 开头的 Bash 命令
- `Bash(npm *)` 匹配任何以 `npm` 开头的命令
- `Bash(* install)` 匹配任何以 `install` 结尾的命令
- `Bash(git * main)` 匹配诸如 `git checkout main` 、 `git merge main` 之类的命令

当 `*` 出现在末尾且前面有空格时（如 `Bash(ls *)` ），它强制执行单词边界，要求前缀后跟空格或字符串结尾。例如， `Bash(ls *)` 匹配 `ls -la` 但不匹配 `lsof` 。相比之下， `Bash(ls*)` 没有空格会同时匹配 `ls -la` 和 `lsof` ，因为没有单词边界约束。

> Claude Code 知道 shell 运算符（如 `&&` ），因此前缀匹配规则如 `Bash(safe-cmd *)` 不会给它权限运行命令 `safe-cmd && other-cmd` 。

> 尝试约束命令参数的 Bash 权限模式很脆弱。例如， `Bash(curl http://github.com/ *)` 旨在将 curl 限制为 GitHub URL，但不会匹配以下变体：
>
> - URL 前的选项： `curl -X GET http://github.com/...`
> - 不同的协议： `curl https://github.com/...`
> - 重定向： `curl -L http://bit.ly/xyz` （重定向到 github）
> - 变量： `URL=http://github.com && curl $URL`
> - 额外空格： `curl http://github.com`
>
> 为了更可靠的 URL 过滤，请考虑：
>
> - **限制 Bash 网络工具**：使用 deny 规则阻止 `curl` 、 `wget` 和类似命令，然后对允许的域使用带有 `WebFetch(domain:github.com)` 权限的 WebFetch 工具
> - **使用 PreToolUse hooks**：实现一个 hook 来验证 Bash 命令中的 URL 并阻止不允许的域
> - 通过 CLAUDE.md 指导 Claude Code 了解您允许的 curl 模式
>
> 请注意，仅使用 WebFetch 不会阻止网络访问。如果允许 Bash，Claude 仍然可以使用 `curl` 、 `wget` 或其他工具来访问任何 URL。

Read 和 Edit

`Edit` 规则适用于所有编辑文件的内置工具。Claude 尽力尝试将 `Read` 规则应用于所有读取文件的内置工具，如 Grep 和 Glob。

Read 和 Edit 规则都遵循 **gitignore** 规范，具有四种不同的模式类型：

Claude Code Docs

模式	含义	示例	匹配
`//path`	从文件系统根目录的**绝对**路径	`Read(//Users/alice/secrets/**)`	`/Users/alice/secrets/**`
`~/path`	从**主**目录的路径	`Read(~/Documents/*.pdf)`	`/Users/alice/Documents/*.pdf`
`/path`	**相对于项目根目录**的路径	`Edit(/src/**/*.ts)`	`<project root>/src/**/*.ts`
`path` 或 `./path`	**相对于当前目录**的路径	`Read(*.env)`	`<cwd>/*.env`

> ⚠ 像 `/Users/alice/file` 这样的模式不是绝对路径。它相对于项目根目录。对于绝对路径，请使用 `//Users/alice/file` 。

示例：

- `Edit(/docs/**)` ：编辑 `<project>/docs/` 中的文件（不是 `/docs/` 也不是 `<project>/.claude/docs/` ）
- `Read(~/.zshrc)` ：读取您主目录的 `.zshrc`
- `Edit(//tmp/scratch.txt)` ：编辑绝对路径 `/tmp/scratch.txt`
- `Read(src/**)` ：从 `<current-directory>/src/` 读取

> ℹ 在 gitignore 模式中， `*` 匹配单个目录中的文件，而 `**` 递归匹配目录。要允许所有文件访问，请使用不带括号的工具名称： `Read` 、 `Edit` 或 `Write` 。

WebFetch

- `WebFetch(domain:example.com)` 匹配对 example.com 的获取请求

MCP

- `mcp__puppeteer` 匹配由 `puppeteer` 服务器提供的任何工具（在 Claude Code 中配置的名称）
- `mcp__puppeteer__*` 通配符语法，也匹配来自 `puppeteer` 服务器的所有工具
- `mcp__puppeteer__puppeteer_navigate` 匹配由 `puppeteer` 服务器提供的 `puppeteer_navigate` 工具

Task（subagents）

Claude Code Docs

使用 `Task(AgentName)` 规则来控制 Claude 可以使用哪些 **subagents**：

- `Task(Explore)` 匹配 Explore subagent
- `Task(Plan)` 匹配 Plan subagent
- `Task(my-custom-agent)` 匹配名为 `my-custom-agent` 的自定义 subagent

将这些规则添加到设置中的 `deny` 数组或使用 `--disallowedTools` CLI 标志来禁用特定代理。要禁用 Explore 代理：

```
{
  "permissions": {
    "deny": ["Task(Explore)"]
  }
}
```

使用hooks扩展权限

提供了一种方式来注册自定义 shell 命令以在运行时执行权限评估。当 Claude Code 进行工具调用时，PreToolUse hooks 在权限系统之前运行，hook 输出可以确定是否批准或拒绝工具调用以代替权限系统。

工作目录

默认情况下，Claude 可以访问启动它的目录中的文件。您可以扩展此访问权限：

- **启动期间**：使用 `--add-dir <path>` CLI 参数
- **会话期间**：使用 `/add-dir` 命令
- **持久配置**：添加到 中的 `additionalDirectories`

其他目录中的文件遵循与原始工作目录相同的权限规则：它们变为可读的而无需提示，文件编辑权限遵循当前权限模式。

权限如何与沙箱交互

权限和**沙箱**是互补的安全层：

Claude Code Docs

- **权限**控制 Claude Code 可以使用哪些工具以及它可以访问哪些文件或域。它们适用于所有工具 ›(Bash、Read、Edit、WebFetch、MCP 和其他)。
- **沙箱**提供操作系统级别的强制执行，限制 Bash 工具的文件系统和网络访问。它仅适用于 Bash 命令及其子进程。

同时使用两者以实现纵深防御：

- 权限 deny 规则阻止 Claude 甚至尝试访问受限资源
- 沙箱限制防止 Bash 命令到达定义边界之外的资源，即使提示注入绕过了 Claude 的决策制定
- 沙箱中的文件系统限制使用 Read 和 Edit deny 规则，而不是单独的沙箱配置
- 网络限制结合 WebFetch 权限规则与沙箱的 `allowedDomains` 列表

托管设置

对于需要对 Claude Code 配置进行集中控制的组织，管理员可以部署无法被用户或项目设置覆盖的托管设置。这些策略设置遵循与常规设置文件相同的格式，可以通过 MDM/操作系统级别策略、托管设置文件或**服务器托管设置**交付。有关交付机制和文件位置，请参见**设置文件**。

仅托管设置

某些设置仅在托管设置中有效：

设置	描述
`disableBypassPermissionsMode`	设置为 `"disable"` 以防止 `bypassPermissions` 模式和 `--dangerously-skip-permissions` 标志
`allowManagedPermissionRulesOnly`	当为 `true` 时，防止用户和项目设置定义 `allow` 、 `ask` 或 `deny` 权限规则。仅应用托管设置中的规则
`allowManagedHooksOnly`	当为 `true` 时，防止加载用户、项目和插件 hooks。仅允许托管 hooks 和 SDK hooks
`allowManagedMcpServersOnly`	当为 `true` 时，仅尊重来自托管设置的 `allowedMcpServers` 。 `deniedMcpServers` 仍然从所有来源合并。请参见**托管 MCP 配置**
`blockedMarketplaces`	市场来源的黑名单。在下载前检查被阻止的来源，因此它们永远不会接触文件系统。请参见**托管市场限制**

Claude Code Docs

设置	描述
`sandbox.network.allowManagedDomainsOnly`	当为 `true` 时，仅尊重来自托管设置的 `allowedDomains` 和 `WebFetch(domain:...)` allow 规则。被拒绝的域仍然从所有来源合并
`strictKnownMarketplaces`	控制用户可以添加哪些插件市场。请参见**托管市场限制**
`allow_remote_sessions`	当为 `true` 时，允许用户启动**远程控制**和**网络会话**。默认为 `true` 。设置为 `false` 以防止远程会话访问

设置优先级

权限规则遵循与所有其他 Claude Code 设置相同的**设置优先级**：托管设置具有最高优先级，其次是命令行参数、本地项目、共享项目和用户设置。

如果权限在用户设置中被允许但在项目设置中被拒绝，则项目设置优先，权限被阻止。

示例配置

此**存储库**包括常见部署场景的启动设置配置。使用这些作为起点并根据您的需要调整它们。

另请参见

- **Settings**：完整的配置参考，包括权限设置表
- **Sandboxing**：Bash 命令的操作系统级文件系统和网络隔离
- **Authentication**：设置用户对 Claude Code 的访问
- **Security**：安全保障和最佳实践
- **Hooks**：自动化工作流并扩展权限评估

此页面对您有帮助吗？ 是 否

‹ 设置 Sandboxing ›

Claude Code Docs
Claude Code Docs

Company

Anthropic

Careers

Economic Futures

Research

News

Trust center

Transparency

Help and security

Availability

Status

Support center

Learn

Courses

MCP connectors

Customer stories

Engineering blog

Events

Powered by Claude

Service partners

Startups program

Terms and policies

Privacy policy

Disclosure policy

Usage policy

Commercial terms

Consumer terms